



Módulo 3 · Programación con Python

Creación de Funciones Simples en Python

Ejercicio de Clase

Nombre del alumno:
Angel David Echenique Villanueva

Fecha: 10/03/26

📖 Instrucciones Responde las preguntas con lo que hemos visto en clase. Lee bien cada parte antes de escribir código. ¡Recuerda probar todo en VS Code!

📖 Objetivo del ejercicio

Este ejercicio te ayudará a practicar:

- Cómo crear funciones en Python
- Cómo usar parámetros y valores de retorno
- Cómo aplicar validaciones sencillas
- Cómo probar tu propio código en VS Code

📖 PARTE 1 — Crear una función que valide si un número es positivo

Instrucciones

1. Crea una función llamada `es_positivo(numero)`
2. La función debe devolver:
 - **True** si el número es mayor que 0
 - **False** en caso contrario
3. Prueba la función con varios valores.

Ejemplo de código

```
def es_positivo(numero):  
    """Devuelve True si el número es mayor que  
    0."""  
    return numero > 0  
  
print(es_positivo(5))      # True  
print(es_positivo(-3))    # False  
print(es_positivo(0))     # False
```

📌 Tu código aquí:

```
def es_positivo(numero):  
    if not isinstance(numero, (int, float)):  
        return "Error: debes ingresar un número"  
    return numero > 0
```

Por esto.

isinstance en Python

1

La función `isinstance` en Python se utiliza para verificar si un objeto es una instancia de una clase o tipo específico. Es especialmente útil para validar tipos de datos y garantizar que los datos se procesen correctamente.

Ejemplo básico:

```
# Verificar tipos básicos  
  
print(isinstance(10, int)) # True  
print(isinstance("Hola", str)) # True  
print(isinstance([1, 2, 3], list)) # True  
print(isinstance(10.5, int)) # False
```



Sintaxis:

```
isinstance(objeto, clase_o_tupla)
```

objeto: El objeto que deseas verificar.

clase_o_tupla: Clase o tupla de clases/tipos contra los cuales se verificará el objeto.

PARTE 2 — Crear una función que capitalice palabras

Instrucciones

1. Crea una función llamada `capitalizar(texto)`
2. La función debe devolver el texto con la primera letra de cada palabra en mayúscula.
3. Si el texto está vacío, debe devolver el mensaje: `"Texto vacío"`

 Pista Puedes usar el método `.title()` de Python.

Ejemplo de código

```
def capitalizar(texto):  
    """Capitaliza la primera letra de cada  
    palabra."""  
    if texto == "":  
        return "Texto vacío"  
    return texto.title()  
  
print(capitalizar("hola mundo"))  
print(capitalizar(""))
```

Tu código aquí:

```
def capitalizar(texto):  
    if not isinstance(texto, str):  
        return "Error: debes ingresar un texto"  
    if texto == "":  
        return "Texto vacío"  
    return texto.title()
```

Esta por la misma razon de la de antes.

PARTE 3 — Validar un email (versión muy simple)

Instrucciones

1. Crea una función llamada `validar_email(email)`
2. La función debe comprobar si el email contiene:
 - Un `"@"`
 - Un `"."`
3. Si cumple ambas condiciones, devuelve `True`.

4. Si no, devuelve **False**.

Ejemplo de código

```
def validar_email(email):  
    """Valida un email de forma muy básica."""  
    return "@" in email and "." in email  
  
print(validar_email("usuario@ejemplo.com")) # True  
print(validar_email("usuario.com"))        # False
```

Tu código aquí:

```
def validar_email(email):  
    if not isinstance(email, str):  
        return "Error: debes ingresar un texto"  
    if "@" not in email:  
        return False  
    if "." not in email:  
        return False  
    return True
```

Igual por la misma razón.

Aunque podría utilizar esta también porque es muy limpia.

```
def validar_email(email):  
    return "@" in email and "." in email
```

PARTE 4 — Mini reto final

Crea una función llamada

Crea una función llamada `resumen_usuario(nombre, email)` que:

1. Capitalice el nombre del usuario
2. Valide el email usando la función anterior
3. Devuelva un mensaje:
 - Si el email es válido: **"Usuario: <Nombre> – Email correcto"**
 - Si el email NO es válido: **"Usuario: <Nombre> – Email inválido"**

Ejemplo de salida esperada

```
Usuario: Ana López — Email correcto
Usuario: Pedro — Email inválido
```

Tu código aquí:

```
def resumen_usuario(nombre, email):
    if not isinstance(nombre, str) or not isinstance(email, str):
        return "Error: datos inválidos"

    nombre_cap = nombre.title()

    if validar_email(email):
        return f"Usuario: {nombre_cap} — Email correcto"
    return f"Usuario: {nombre_cap} — Email inválido"

Ejemplos
print(resumen_usuario("kindred", "kindred@spirit.com"))
print(resumen_usuario("yasuo", "yasuo_sin_arroba.com"))
```

 Entrega El alumnado debe entregar un archivo .py con las cuatro funciones y las pruebas realizadas.